KHULNA UNIVERSITY OF ENGINEERING AND TECHNOLOGY

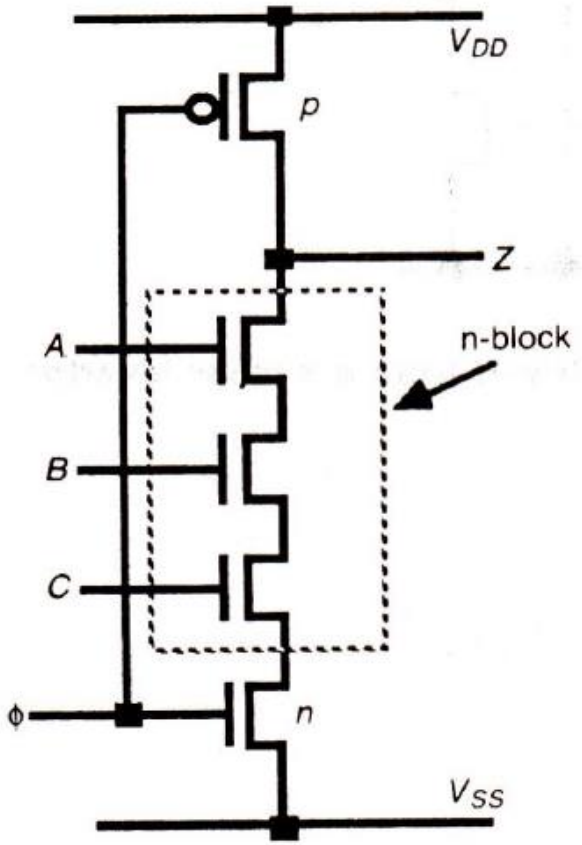
Courtesy:
Dr. Md. Selim Hossain

Department of Electrical and Electronic Engineering

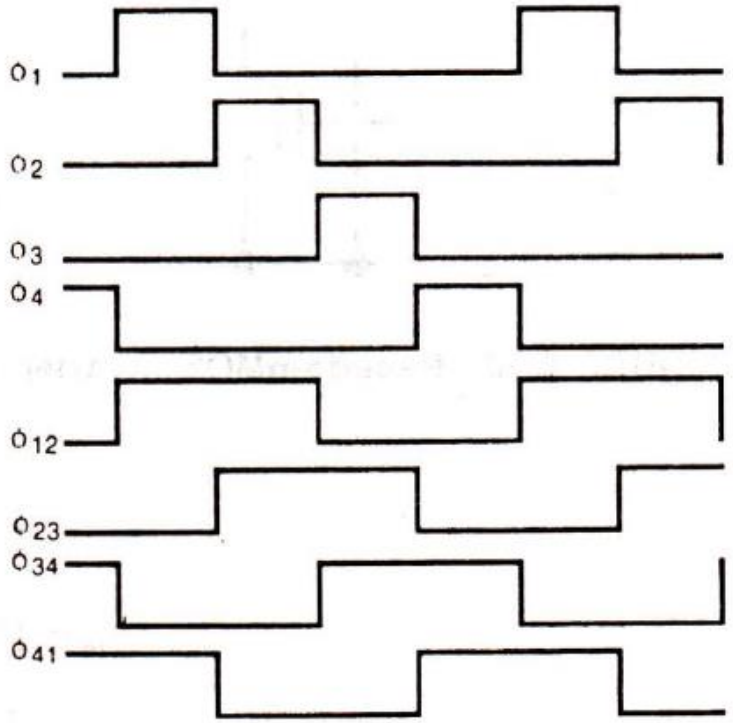
Course Code : EE 4121

VLSI

Dynamic CMOS logic



(a) Schematic

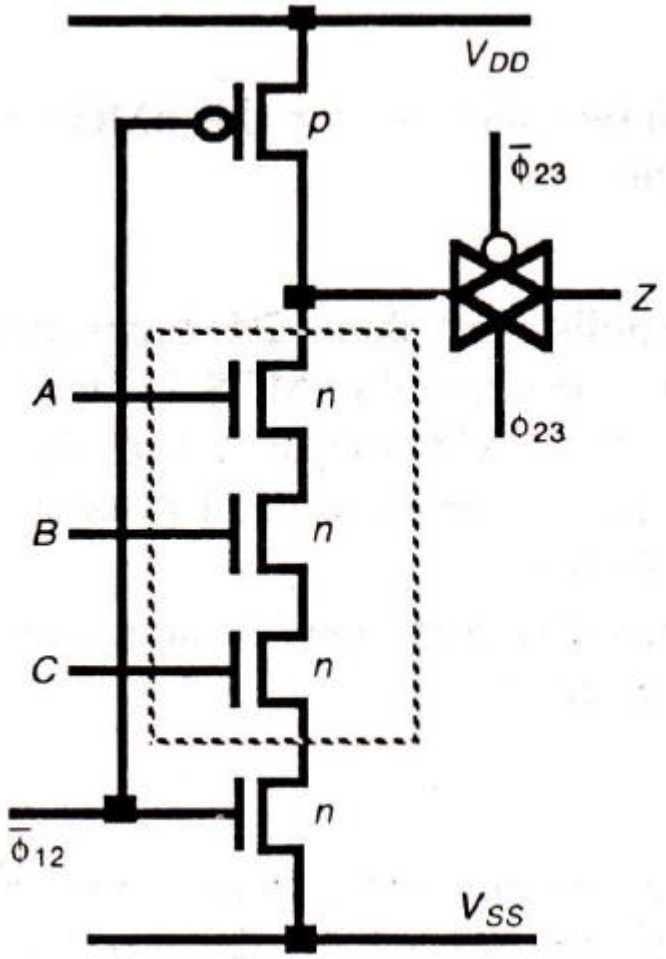


(b) Possible 4o and derived clocks

- pMOS is used for precharing.
- Additional clock is used in CMOS
- Off period of clock => Pre-charging or ϕ = 0, then Z = Vdd, pre-charging and capacitor charged.
- ON period of clock=> Evaluation

Note: (1) Charge sharing may be a problem unless the inputs are constrained not to change during the on period of the clock.

(2) Single phase dynamic logic structures cannot be cascaded since, owing to circuit delays, an incorrect input to the next stage may be present when evaluation begins, so that its output is inadvertently discharged and the wrong output results.



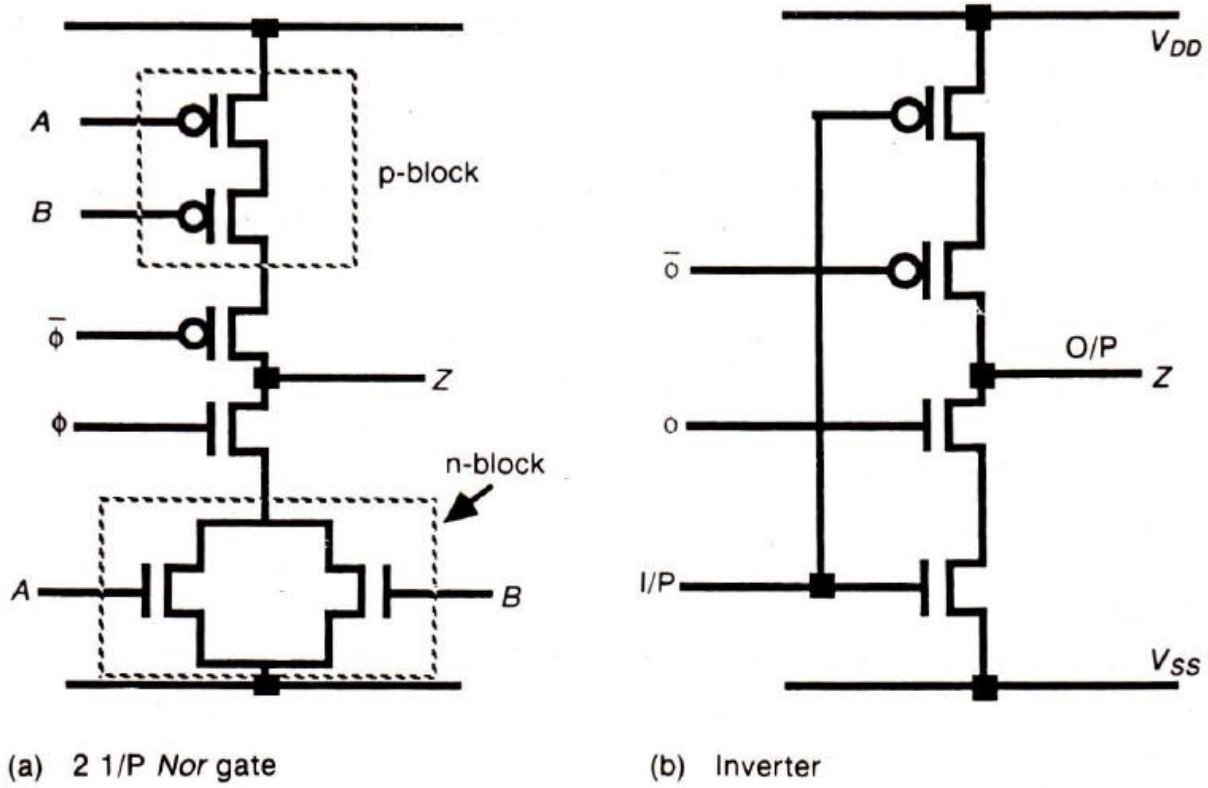
(c) Type 3 arrangement

TABLE 6.1 Dynamic logic types and sequences

Gate type	*Evaluate clock*	*Transmission gate clock*	*Allowable next types*
Type 1	$\overline{\phi}_{34}$	ϕ_{41}	Types 2 or 3
Type 2	$\overline{\phi}_{41}$	ϕ_{12}	Types 3 or 4
Type 3	$\overline{\phi}_{12}$	ϕ_{23}	Types 4 or 1
Type 4	$\overline{\phi}_{23}$	ϕ_{34}	Types 1 or 2

One remedy is to employ a four-phase clock in which the actual signals used are the derived clocks ϕ_{12}, ϕ_{23}, ϕ_{34}, and ϕ_{41}, as illustrated in Figure 6.11(b).

The basic circuit of Figure 6.11(a) is modified by the inclusion of a transmission gate as in Figure 6.11(c), the function of which is to sample the output during the 'evaluate' period and to hold the output state while the next stage logic evaluates. For this strategy to work, the next stage must operate on overlapping but later clock signals. Clearly, since there are four different derived clock signals which are used in sequential pairs (e.g. ϕ_{12} and ϕ_{23} in Figure 6.11(c)), there are four different gate clocking configurations. These configurations are usually identified by a type number which reflects the last of the clock periods activating the gate. For example, the gate shown would be identified as 'type 3' since the output Z is precharged during ϕ_2 and is evaluated during ϕ_3 (the transmission gate is clocked by ϕ_{23}). In order to avoid erroneous evaluations, the gates must be connected in allowable sequences as set out in Table 6.1.



(a) 2 1/P *Nor* gate (b) Inverter

FIGURE 6.12 Clocked CMOS (C^2MOS) logic.

6.3.4.3 Clocked CMOS (C^2MOS) logic

The general arrangement may be made clearer by Figure 6.12. The logic is implemented in both n- and p-transistors in the form of a pull-up p-block and a complementary n-block pull-down structure (Figure 6.12(a)), as for the inverter-based CMOS logic discussed earlier. However, the logic in this case is evaluated (connected to the output) only during the on period of the clock. As might be expected, a clocked inverter circuit forms part of this family of logic as shown in Figure 6.12(b). Owing to the extra transistors in series with the output, slower rise-times and fall-times can be expected.

6.3.4.4 CMOS domino logic

An extension to the dynamic CMOS logic discussed earlier is set out in Figure 6.13. This modified arrangement allows for the cascading of logic structures using only a single phase clock. This requires a static CMOS buffer in each logic gate.

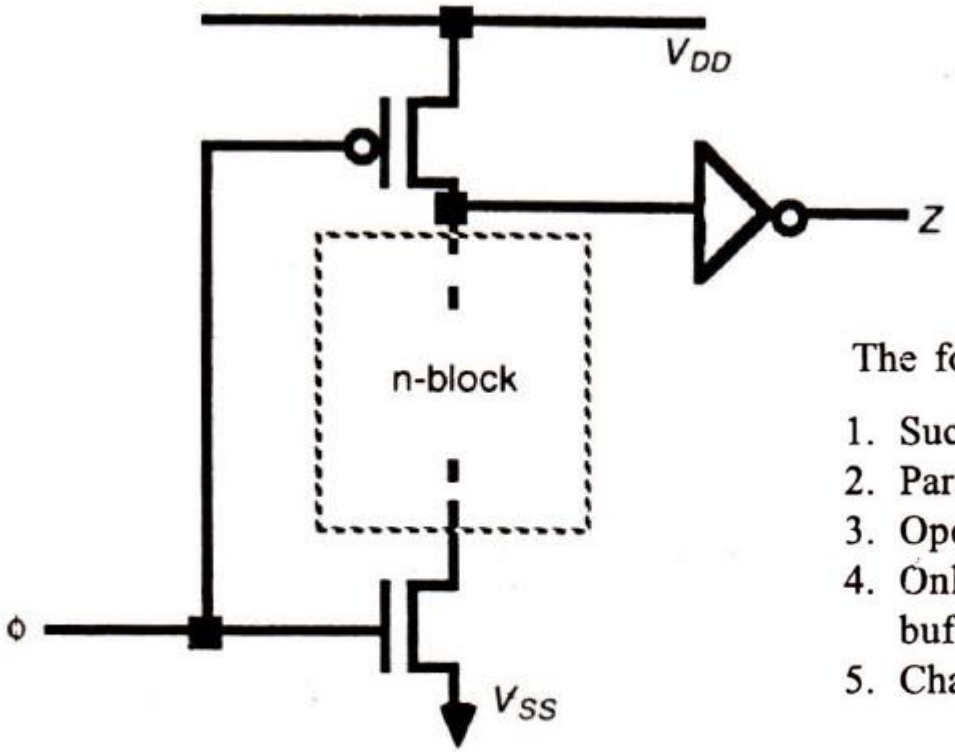


FIGURE 6.13 CMOS domino logic.

The following remarks will help to place this type of logic in the scheme of things:

1. Such logic structures can have smaller areas than conventional CMOS logic.
2. Parasitic capacitances are smaller so that higher operating speeds are possible.
3. Operation is free of glitches since each gate can make only one '1' to '0' transition.
4. Only non-inverting structures are possible because of the presence of the inverting buffer.
5. Charge distribution may be a problem and must be considered.

6.3.4.5 n-p CMOS logic

This is another variation of basic dynamic logic arrangement, in which the actual logic blocks are alternately 'n' and 'p' in a cascaded structure as in Figure 6.14. The precharge and evaluate transistors are fed from the clock ϕ and clockbar $\overline{\phi}$ alternately, and clearly the functions of the top and bottom transistors also alternate, between precharge and evaluate.

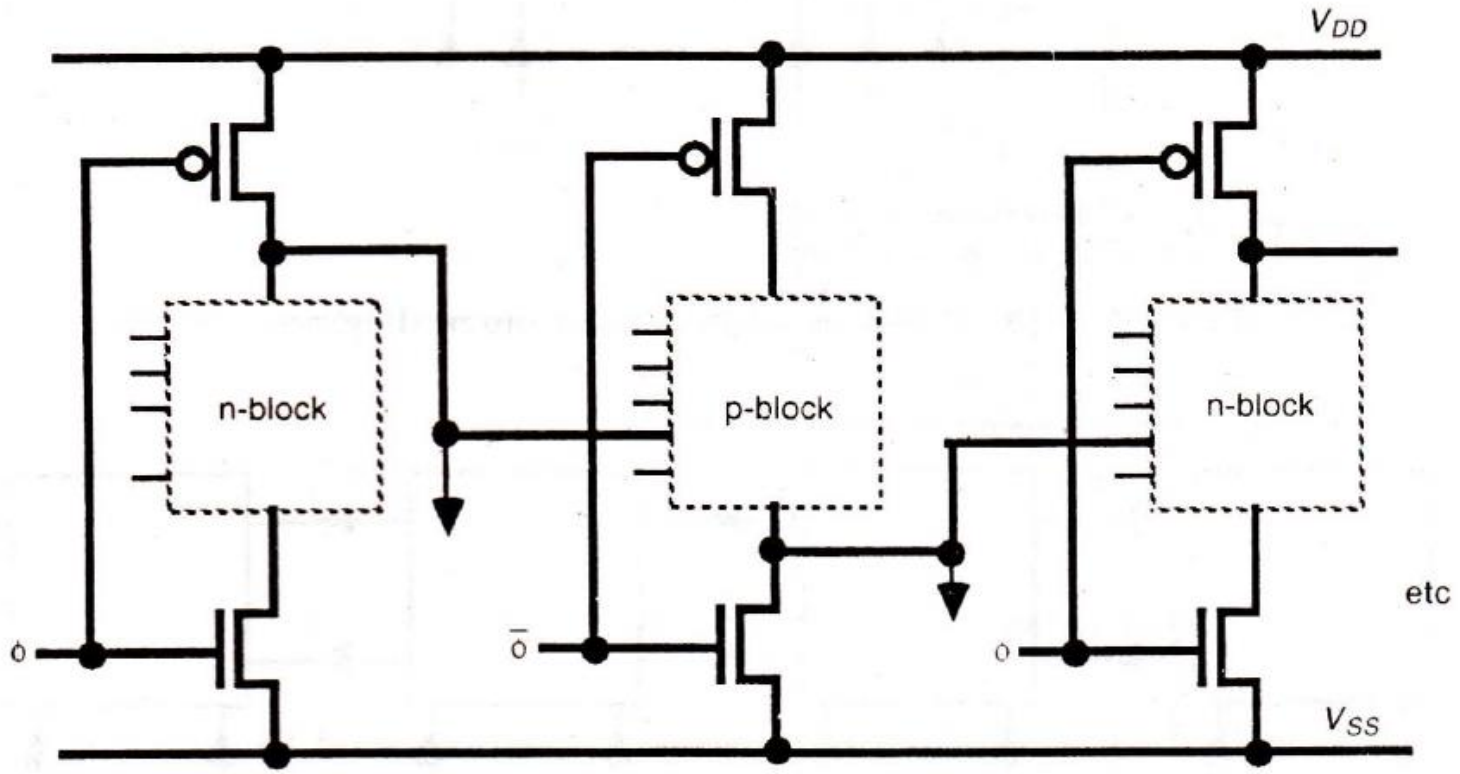


FIGURE 6.14 n-p CMOS logic.

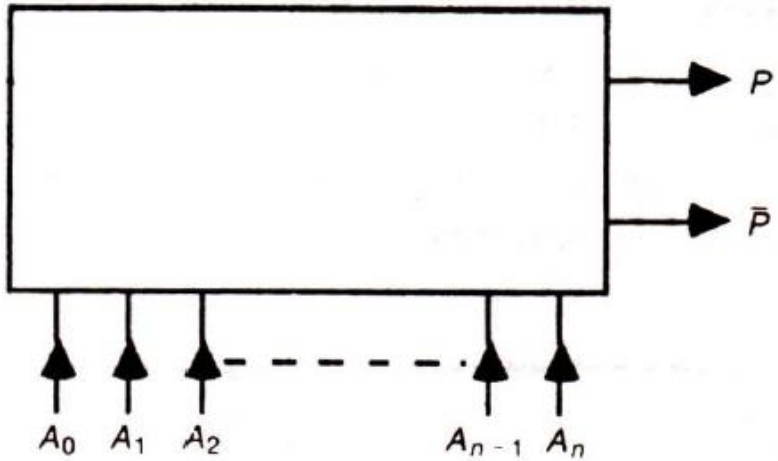
Other forms of CMOS logic are also possible, but this text does not attempt to give an exhaustive treatment.

Structural Design (Combinational Logic)—Parity Generator

A circuit is to be designed to indicate the parity of a binary number or word. The requirement is indicated in Figure 6.15 for an $(n + 1)$-bit input.

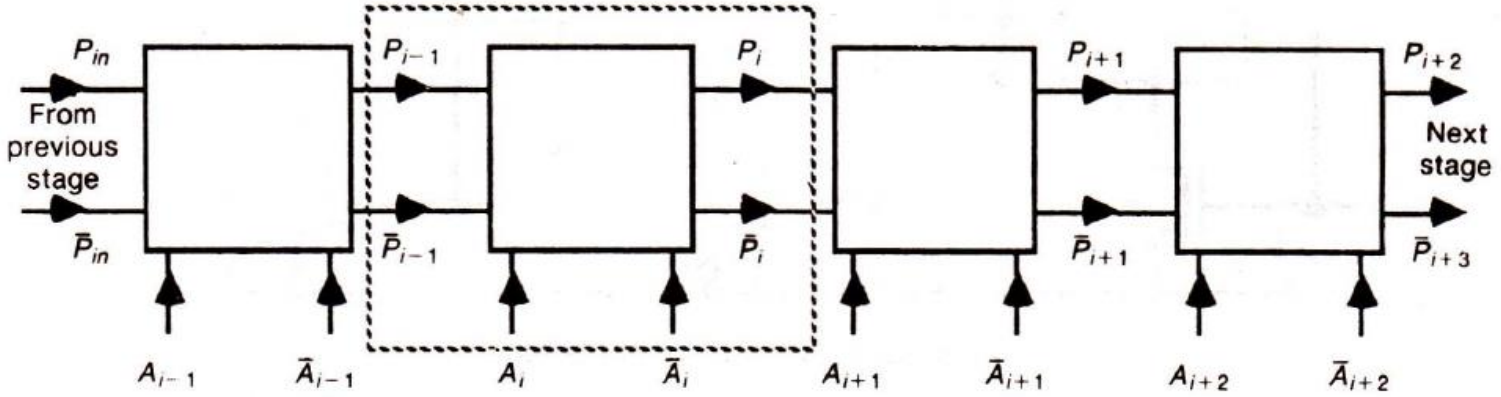
Since the number of bits is undefined, we must find a general solution on a cascadable bit-wise basis so that n can have any value. A suitably regular structure is set out in Figure 6.16. From this, we may recognize a standard or basic one-bit cell from which an n-bit parity generator may be formed. Such a cell is shown in Figure 6.17.

It will be seen that parity information is passed from one cell to the next and is modified or not by a cell, depending on the state of the input lines A_i and $\bar{A}_i$.



Note: $P = \begin{cases} 1 & \text{Even number of 1s at input} \\ 0 & \text{Odd number of 1s at input} \end{cases}$

FIGURE 6.15 Parity generator basic block diagram.



Note: Parity requirements are set at the left-most cell where $P_{in} = 1$ sets even and $P_{in} = 0$ sets odd parity.

FIGURE 6.16 Parity generator—structured design approach.

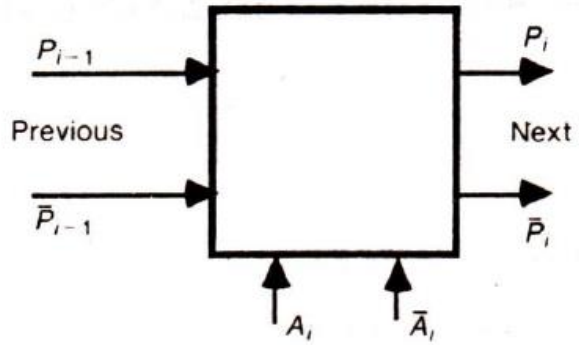


FIGURE 6.17 Parity generator—basic one-bit cell.

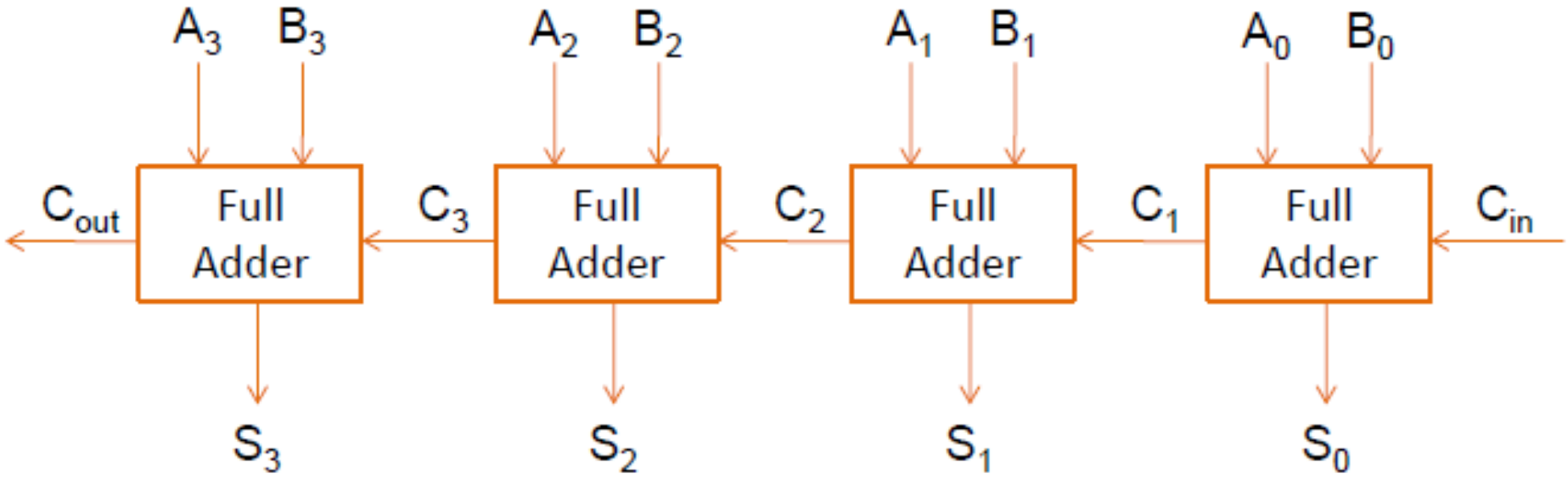
A little reflection will readily reveal that the requirements are:

$A_i = 1$ parity is changed, $P_i = \bar{P}_{i-1}$
$A_i = 0$ parity is unchanged, $P_i = P_{i-1}$

A suitable arrangement for such a cell is given in stick diagram form in Figure 6.18(a) (nMOS) and 6.18(b) (CMOS). The circuit implements the function

$$P_i = \bar{P}_{i-1} . A_i + P_{i-1} . \bar{A}_i$$

Structural Design (Combinational Logic)—4-bit Adder




```
FA0: Full_Adder port map (A(0),B(0),Cin,C(1),S(0));

FA1: Full_Adder port map (A(1),B(1),C(1),C(2),S(1));

FA2: Full_Adder port map (A(2),B(2),C(2),C(3),S(2));

FA3: Full_Adder port map (A(3),B(3),C(3),Cout,S(3));
```

Bus Arbitration Logic for n-line Bus

The requirement can be expressed algebraically as follows:

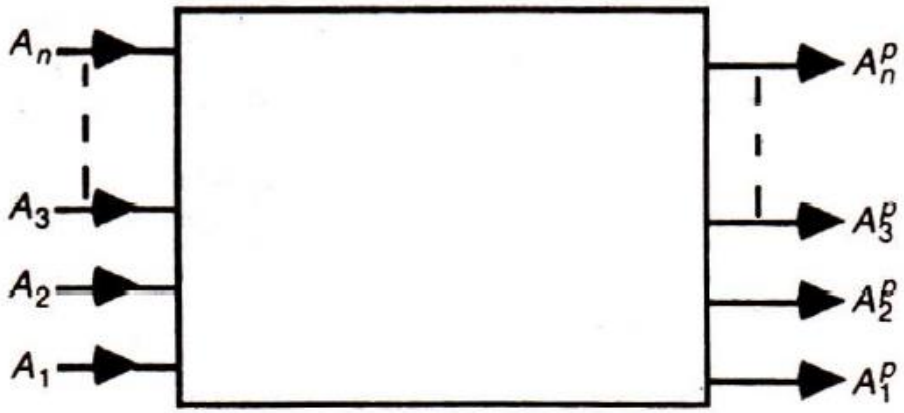
$$A_n^p = A_n$$

$$A_{n-1}^p = \bar{A}_n \cdot A_{n-1} \qquad [\bar{A}_{n-1}^p = A_n + \bar{A}_{n-1}]$$

$$A_{n-2}^p = \bar{A}_n \cdot \bar{A}_{n-1} \cdot A_{n-2} \qquad [\bar{A}_{n-2}^p = A_n + A_{n-1} + \bar{A}_{n-2}]$$

$$\vdots \qquad\qquad \vdots$$

$$A_n^p = \bar{A}_n \cdot \bar{A}_{n-1} \cdot \bar{A}_{n-2} \quad \ldots\ldots \quad \bar{A}_3 \cdot \bar{A}_2 \cdot A_1 \quad \text{(etc.)}$$



Truth table

A_n	.....	A_3	A_2	A_1	A_n^p	.....	A_3^p	A_2^p	A_1^p
0	.....	0	0	0	0	.....	0	0	0
0		0	0	1	0	.....	0	0	1
0		0	1	X	0		0	1	0
0		1	X	X	0		1	0	0
.		.	.	.	.		.	.	.
.		.	.	.	.		.	.	.
.		.	.	.	.		.	.	.
1	.....	X	X	X	1	.....	0	0	0

A direct but unstructured implementation of these expressions may be readily envisaged and a suitable arrangement of switch (pass transistor) logic is given in Figure 6.20.

This implementation seems the obvious one, but it does suffer from the fact that as the input line under consideration moves down in significance so the complexity of the logic grows. For example, we have shown only the top three lines in Figure 6.20, but it will be seen that:

A_n requires one diffusion path and no switches
A_{n-1} requires two diffusion paths and two switches
A_{n-2} requires three diffusion paths and four switches

and so on.

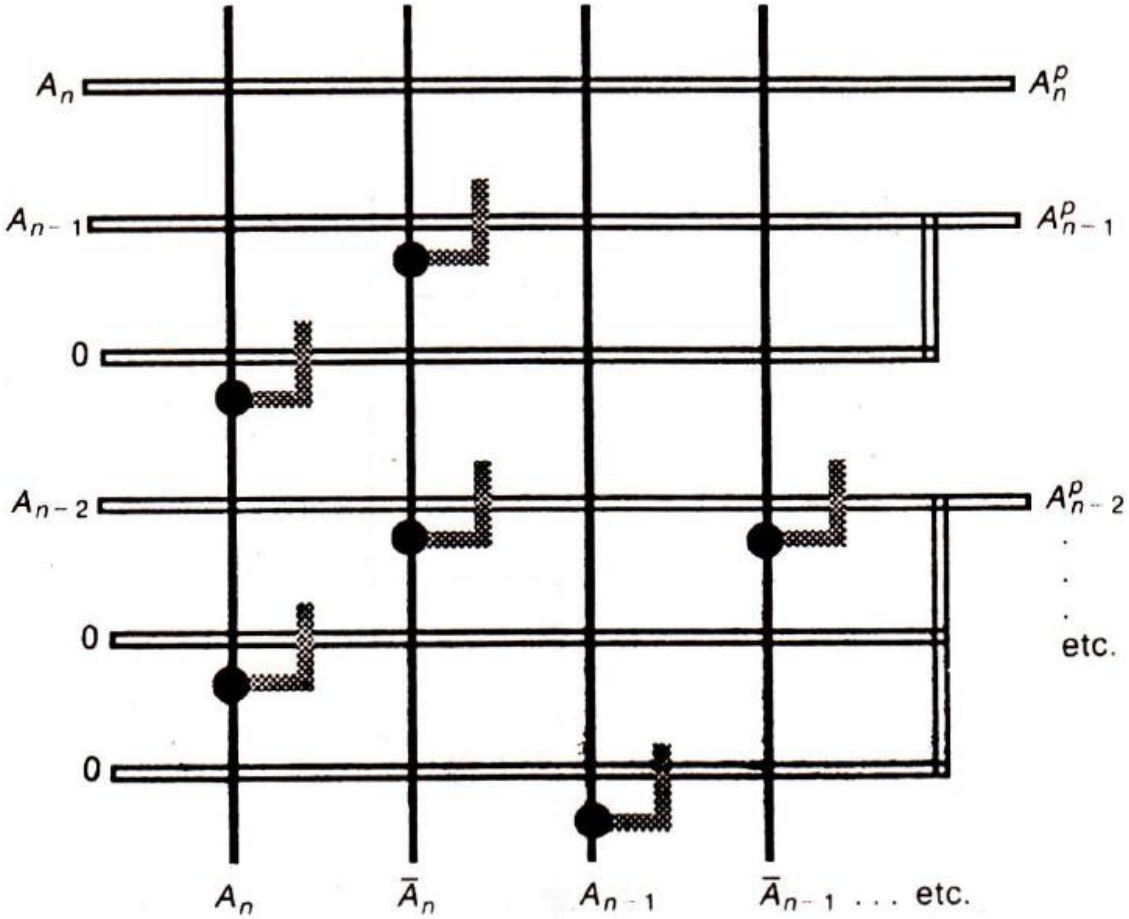


FIGURE 6.20 Stick diagram–bus arbitration logic.

Structural Design using Grant Line

A regular structure having been arrived at, the requirements for each cell may be expressed as follows:

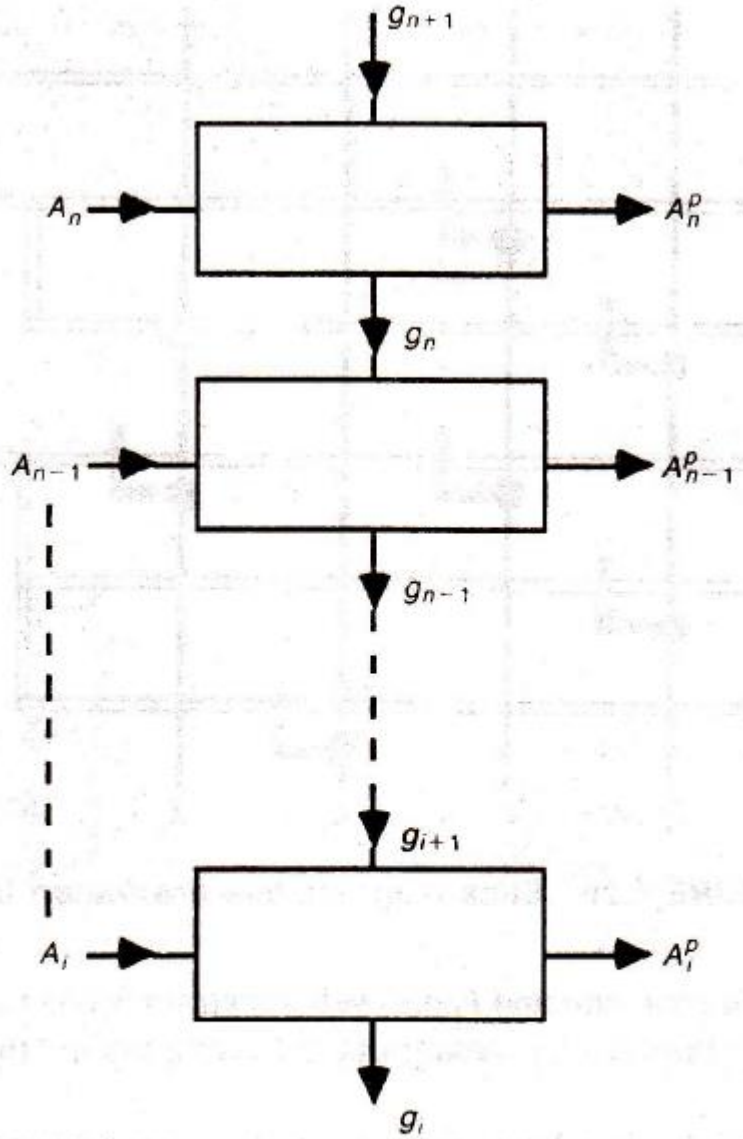
$$A_i^p = \begin{cases} g_{i+1} & \text{if } A_i = 1 \\ \text{or } 0 & \text{otherwise} \end{cases}$$

$$g_i = \begin{cases} 0 & \text{if } A_i = 1 \\ \text{or } g_{i+1} & \text{otherwise} \end{cases}$$

These requirements may be met by the circuit of Figure 6.22, but care must be taken not to cascade more than four cells without buffering the grant line.

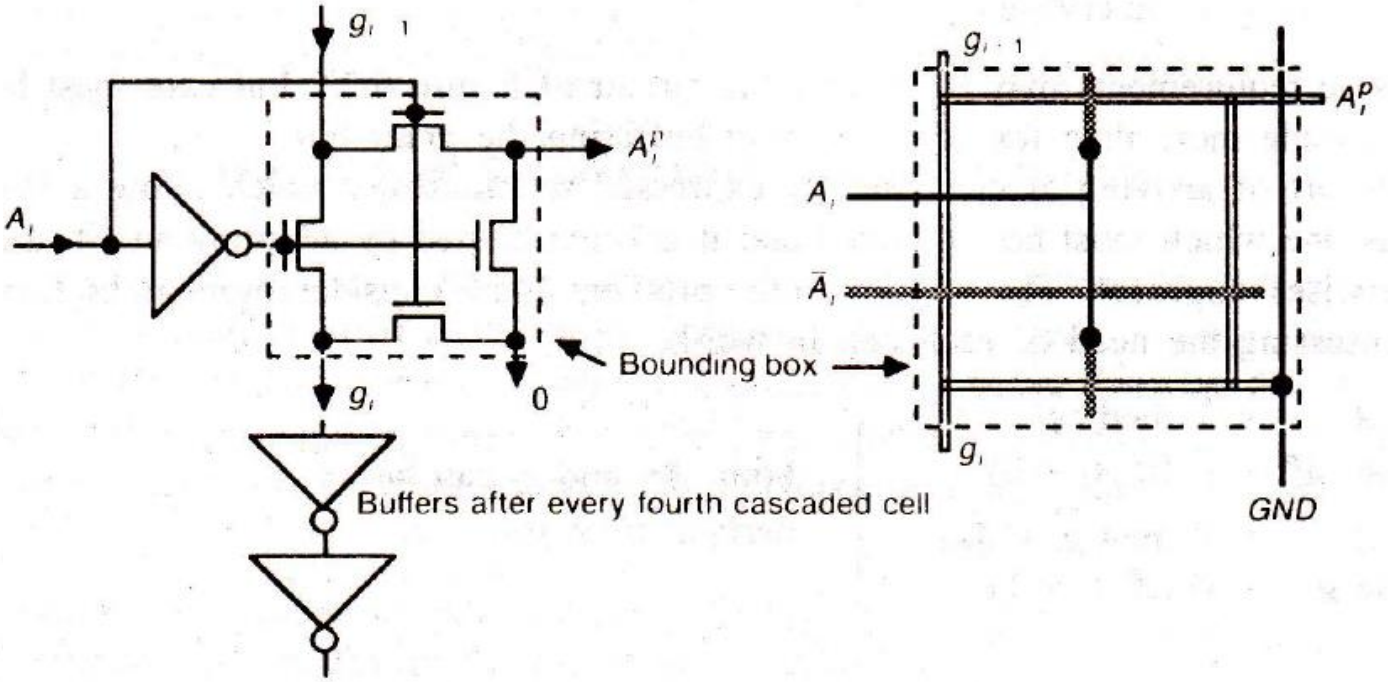
If $A_i = 1$ then $A_i^p = g_{i+1}$,
else $A_i^p = 0$ (if $A_i = 0$)
If $A_i = 0$ then $g_i = g_{i+1}$
else $g_i = 0$ (if $A_i = 1$)

} both A_i^p and g_i can be derived from g_{i+1}



Notes: 1. g = grant line.
2. If grant line is 1, none of the lines above it wants priority.
3. If $A_i = 0$, pass grant.

FIGURE 6.21 Bus arbitration logic—structured design.



(a) Circuit (b) Stick diagram

FIGURE 6.22 Bus arbitration logic—structured design.

From which we could deduce

$$A_i^p = A_i \cdot g_{i+1}$$

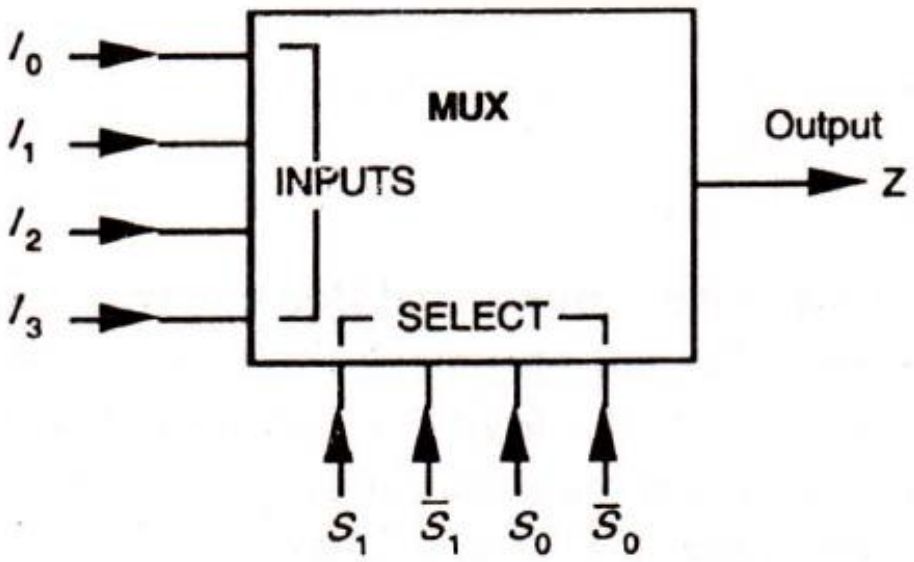
$$g_i = \bar{A}_i \cdot g_{i+1}$$

$$A_i^p = A_i \cdot g_{i+1}\,;\; \bar{A}_i^p = \bar{A}_i + \bar{g}_{i+1}$$

$$g_i = \bar{A}_i g_{i+1}\,;\; \bar{g}_i = A_i + \bar{g}_{i+1}$$

Multiplexer

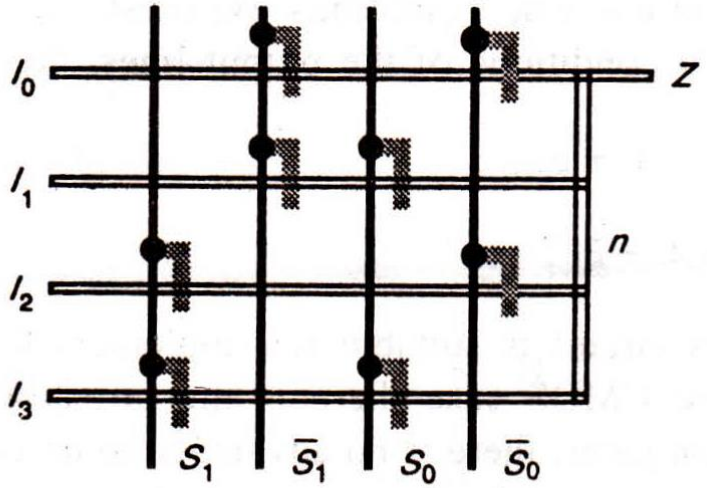
(Data Selectors)



Truth table

S_1	S_0	Z
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

FIGURE 6.23 Selector logic circuit.

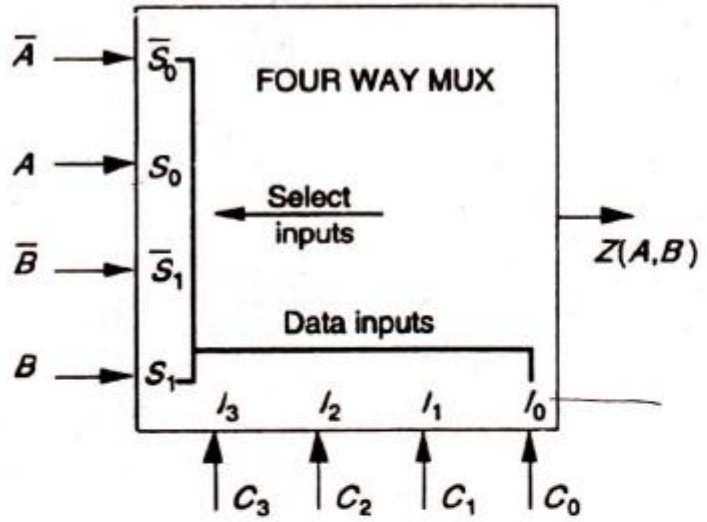


(a) nMOS switches

$$Z = I_0.\overline{S}_1.\overline{S}_0 + I_1.\overline{S}_1.S_0 + I_2.S_1.\overline{S}_0 + I_3.S_1.S_0$$

FIGURE 6.24 Switch logic implementations of a four-way multiplexer.

A General Logic Function Block (MUX as Programmable Functional Ckt)



INPUT PROGRAMMING C_3	C_2	C_1	C_0	FUNCTION $Z(A,B)$	
0	0	0	0	0	$Z=0$
0	0	0	1	$\overline{A}.\overline{B}$; $\overline{A+B}$	*Nor*
0	0	1	0	$A.\overline{B}$	
0	0	1	1	$\overline{B}$	Not *B*
0	1	0	0	$\overline{A}.B$	
0	1	0	1	$\overline{A}$	Not *A*
0	1	1	0	$A.\overline{B}+\overline{A}.B$	*Exclusive-Or*
0	1	1	1	$\overline{A}+\overline{B}$; $\overline{AB}$	*Nand*
1	0	0	0	$A.B$	*And*
1	0	0	1	$\overline{A}.\overline{B}+A.B$	Comparator
1	0	1	0	A	$O/P=A$
1	0	1	1	$A+\overline{B}$	
1	1	0	0	B	$O/P=B$
1	1	0	1	$\overline{A}+B$	
1	1	1	0	$A+B$	*Or*
1	1	1	1	1	$Z=1$

FIGURE 6.26 General logic function block (two variables).